JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2302 SOFTWARE ENGINEERING 1

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 8

Software Maintenance

Learning Outcomes

A study of this lesson should enable students to:

- Describe software maintenance
- Describe the types of software maintenance

Software maintenance is the general process of changing a system after it has been delivered. The changes maybe simple changes to correct coding errors, more extensive changes to correct design errors or accommodate new requirements. Software maintenance does not normally involve major architectural changes to the system.

Changes are implemented by modifying existing system components and where necessary by adding new components to the system.

There are three different types of software maintenance:

1. Maintenance to repair software faults coding errors are usually relatively cheap to correct; design errors are more expensive as they may involve the rewriting of several program components. Requirements errors are the most

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

expensive to repair because of the extensive system redesign which may be necessary.

2. Maintenance to adapt the software to a different operating environment this type of maintenance is required when some aspect of the system's environment such as the hardware, the platform operating system or other support software changes. The application system must be modified to adapt it to cope with these environmental changes.

3. Maintenance to add or modify the system's functionality this type of maintenance is necessary when the system requirements change in response to organizational or business change. The scale of the changes required to the software is often much greater than for the other types of maintenance.

In practice there is not a clear cut distinction between these different types of maintenance. Software faults may be revealed because a system has been used in an unanticipated way and the best way to repair these faults may be to add new functionality to help users with the system. When adapting the software to new environment, functionality maybe added to take advantage of new facilities supported by the environment. Adding new functionality to a system may be necessary because faults have changed the usage patterns of the system a side effect of the new functionality is to remove the faults from the software.

While these different types of maintenance are generally recognized, different people sometimes give them different names. Corrective maintenance is used to refer to maintenance for fault repair. However, adaptive maintenance sometimes means adaptation

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

ing to a new environment and sometimes means adapting software to new requirements. Perfective maintenance sometimes means perfecting the software by implementing new requirements and, in other cases, maintaining the functionality of the system by improving its structure and its performance. The relative effort devoted to the different types of maintenance is as follows:

- Corrective maintenance - 17 %
- Adaptive maintenance - 18%
- Perfective maintenance - 65%

From these figures we can see that repairing system faults is not the most expensive maintenance activity. Rather, evolving the system to cope with new environments and new or changed requirements consumes most maintenance effort. Maintenance is

therefore a natural continuation of the system development process with associated specification, design, implementation and testing activities.

The cost of system maintenance represents a large proportion of the budget of most organizations that use software systems. Maintenance cost as a proportion of development costs vary from one application domain to another. One important reason why maintenance costs are high is that it is more expensive to add functionality after a system is in operation than it is to implement the same functionality during development. The key factors that distinguish development and maintenance and which lead to higher maintenance costs are:

1. Team Stability - After a system has been delivered, it is normal for the development team to be broken up and peo-

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

ple to work on new projects. The new team or individuals responsible for system maintenance do not understand the system or the background to the system design decisions. A lot of the effort during the maintenance process is taken up with understanding the existing system before implementing changes to it.

2. Contractual Responsibility - The contract to maintain a system is usually separate from the system development contract. The maintenance may be given to a different company rather than the original system developer. This factor, along with the lack of team stability, means that there is no incentive for a development team to write the software so that it is easy to change. If a development team can cut corners to save effort during development it

is worthwhile for them to do so even if it means increasing maintenance costs.

3. Staff Skills - Maintenance staff are often relatively inexperienced and unfamiliar with the application domain. Maintenance has a poor image among software engineers. It is seen as a less skilled process than system development and is often allocated to the most junior staff. Furthermore, old systems maybe written in obsolete program languages. The maintenance staff may not have much experience of development in these languages to maintain the system.

4. Program age and structure - as programs age, their structures tend to be degraded by change and so they become harder to understand and change. Furthermore, many legacy systems have been developed without modern soft-

ware engineering techniques. They were never well structural and they were often optimized for efficiency rather than understandability. The documentation for old systems may not have been subject to configuration management, so time is often wasted finding the right versions of the system components to change.

The first 3 of these problems stem from the fact that many organizations still make a distinction between system development and maintenance. The only long term solution to this problem is to accept that systems rarely have a defined lifetime but continue in use, in some form, for an indefinite period.

Rather than develop systems, maintain them until further maintenance is impossible and then replace them, we have to adopt the notion of evolutionary systems. Evolutionary systems

are systems that are designed to evolve and change in response to new demands. They can be created from existing legacy systems by improving their structure through re-engineering and by evolving the architecture of these systems.

The maintenance process

Maintenance processes vary considerably depending on the type of software being maintained, the development processes used in an organization and the people involved in the process. The maintenance process is triggered by a set of change requests from system users, management or customers. The cost and impact of these changes are assessed to see how much of the system is affected by the change and how much it might cost to implement the change. If the proposed changes are accepted, a new

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

release of the system is planned. During the release planning, all proposed changes are considered. A decision is then made on which changes to implement in the next version of the system. The changes are implemented and validated and a new version of the system is released. The process then iterates with a new set of changes proposed for the new release. The figure8.1 below shows an overview of this process.

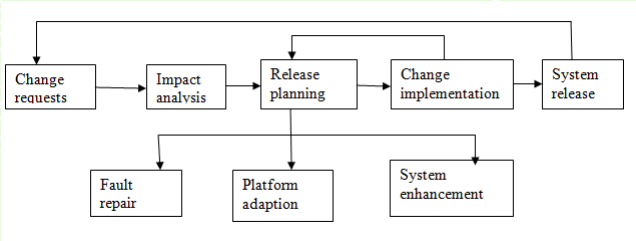
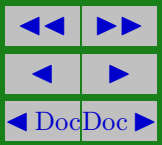


Figure 8.1: An overview of the maintenance process



Revision Questions

EXERCISE 1. ✍ Define Software Maintenance

Example ✎. Describe the types of software maintenance

Solution:

Corrective Maintenance - corrective maintenance means a REACTIVE modification, done in the software product after the delivery . The purpose of corrective maintenance is to correct / or fix discovered (or user reported) problems in the system .

Adaptive Maintenance - adaptive maintenance is also a modification done after delivery, in order to keep the software product usable in changing (or changed) environments / business environments . If this is not done properly by the time of

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

change, business opportunities will be lost .

Perfective Maintenance - a software should be efficient / less resource consuming / and should be easy to cope with . Perfective maintenance is done in order to improve the software performance (after a change in the software or / the environment, the performance of the software changes) . Also improving the maintainability is a concern .

Preventive Maintenance - of course as it sounds, it's just PREVENTING . Preventive Maintenance is done, to detect and correct latent (not developed) faults before they become effective faults . This simply means the prevention of future problems .

References and Additional Reading Materials

1. Hans van Vliet (2005). Software Engineering: Principles and Practice (2nd ed.). John Wiley & Sons, Inc. ISBN: 0-471-97508-7
2. Ian Sommerville, Pete Sawyer(2005). Requirements Engineering: A Good Practice Guide. ISBN: 0-471-97444-7. John Wiley & Sons, Inc
3. Gerald Kotonya, Ian Sommerville (1998). Requirements Engineering: Processes and Techniques. John Wiley & Sons, Inc. ISBN: 0-471-97208-8

Solutions to Exercises

Exercise 1.

is the activity of modifying the software product after delivery , in order to correct faults, improve performance and to improve other attributes (attributes of a good software) . Exercise 1